



Matricule N°: _____ - _____ - _____.

Nom : _____.

Prénom : _____.

Programme : _____.

GIF-1003– Programmation avancée en C++

Examen final Hiver 2021

Durée: 2h50

Aucun document autorisé

Lisez l'examen en entier avant de le commencer. Attention, chaque détail compte.

- Ce questionnaire comporte 4 parties sur 15 pages (incluant une annexe).
- Veuillez répondre dans les espaces prévus sur le questionnaire.
- Soyez concis dans vos réponses, les espaces devant être suffisants pour contenir vos réponses. Vous pouvez vous servir du verso comme brouillon.

1. Principes (12 points)

Test unitaire (4 points)

Quel est l'objectif des tests des cas valides?

Quel est l'objectif des tests des cas invalides?

Pour la correction	p1	
	p2	
	p3	
	p4	
	p5	
	p6	
	p7	
	p8	
	p9	
	p10	
	p11	
	p12	
	p13	
	Total	

Hérarchie de classe, Héritage et polymorphisme (4 points)

Soit la hiérarchie de classe suivante:

```
#include <string>
```

```
class Base
{
public:
    Base (std::string p_base);
    virtual ~Base(){};
private:
    std::string m_b;
};
class Derivee: public Base
{
public:
    Derivee(std::string p_base, std::string p_derive);
private:
    std::string m_d;
};
```

Pourquoi a-t-on défini un destructeur virtuel dans la classe de base, même s'il n'y a aucune ressource particulière à gérer, soit que sa définition ne contient aucune instruction? Expliquez à l'aide d'un exemple de code.

En C++, par quels moyens et sous quelles conditions peut-on faire des traitements polymorphes sur un ensemble d'objets?

Gestion de la mémoire (4 points)

Quand est-ce qu'une classe est dite de forme canonique de Coplien ? Qu'est-ce que cela implique?

Quelles sont les conséquences des fuites de mémoire sur l'exécution d'un programme?

Application

Nous voulons développer un programme afin de gérer un annuaire. Ce programme est construit au fur et à mesure. Sa construction est guidée par une série de questions indépendantes.

Dans tout le développement, la théorie du contrat doit être appliquée (il ne sera pas toujours précisé explicitement dans les questions qui suivent qu'il faut le faire. Alors pensez – y systématiquement.

Pour cela vous disposez des macros définies dans le fichier `ContratException.h`:

`PRECONDITION(test logique)`

`INVARIANT(test logique)`

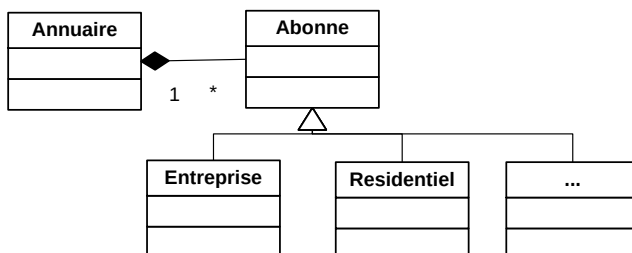
`POSTCONDITION(test logique)`

`INVARIANTS()`

`INVARIANTS()` appelle la méthode `verifieInvariant` que vous allez devoir implémenter.

2. Hiérarchie de classe, Héritage et polymorphisme

Les données manipulées correspondent à différents types d'abonné. Voici la hiérarchie de classe utilisée :



* La classe Residentiel n'est pas développée ici

Toutes les interfaces de ces classes sont données, mais ne sont pas tout à fait complètes. **Il faut les compléter au fur et à mesure si besoin.**

Ces classes seront implémentées dans l'espace de nom ***examenFinal***

Certaines méthodes sont déclarées dans ces interfaces, mais ne sont pas à implémenter, supposant qu'elles le sont déjà. Pensez à les utiliser au besoin.

N'oubliez pas d'ajouter les mots clés comme const, virtual, etc. lorsque c'est requis !

Interface de la classe Abonne (4 points):

```
// Fichier:      Abonne.h
#ifndef ABONNE_H_DEJA_INCLU
#define ABONNE_H_DEJA_INCLU

#include <string>

class Abonne
{
public:

    virtual ~Abonne() {} ;

    const std::string& reqNom() const;
    const std::string& reqTelephone() const;
    const std::string& reqAdresse() const;
    std::string reqAbonneFormate() ;

    bool operator==(const Abonne& p_abonne) const;

private:

    std::string m_nom; // ne doit pas être vide
    std::string m_telephone; // doit être validé par util::validerTelephone
    std::string m_adresse; // aucune contrainte
};

#endif // --- #ifndef ABONNE_H_DEJA_INCLU
```

Interface de la classe Entreprise (6 points):

```
// Fichier:      Entreprise.h
#ifndef ENTREPRISE_H_DEJA_INCLU
#define ENTREPRISE_H_DEJA_INCLU

#include <string>

class Entreprise
{
public:

    Abonne* clone()      ;

    const std::string& reqSiteWeb() const;
    const std::string& reqTelecopie() const;

private:

    std::string m_siteWeb; //aucune contrainte
    std::string m_telecopie; // doit être validé par util::validerTelephone
};

#endif // --- #ifndef ENTREPRISE_H_DEJA_INCLU
```

Interface de la classe Annuaire (7 points):

```
// Fichier:      Annuaire.h
#ifndef ANNUAIRE_H_DEJA_INCLU
#define ANNUAIRE_H_DEJA_INCLU

#include <vector>
#include <stdexcept>
#include <string>

class Annuaire
{
public:
    Annuaire(const std::string& p_ville);

    ~Annuaire();

    std::string reqAnnuaireFormate() const;

private:
    std::string m_ville;
};

#endif // --- #ifndef ANNUAIRE_H_DEJA_INCLU
```

Pour la classe Abonne (9 points)

On vous demande d'écrire les fonctions membre suivantes :

- un constructeur avec paramètres. Les paramètres sont le nom, le numéro de téléphone et l'adresse. Les contraintes sur les attributs sont rappelées dans l'interface (pensez au contrat). Prévoir ici l'entête de commentaires de spécification qui seront utilisés pour générer la documentation avec Doxygen (voir l'annexe pour les balises). Cet en-tête doit être complet.

[illegible]

- La fonction `reqAbonneFormate` qui doit être implantée **systématiquement** dans toute classe dérivée de la classe `Abonne`. Faites une déclaration en conséquence dans l'interface de la classe. Il n'est pas demandé de l'implémenter. Cette méthode retourne un objet string avec mise en forme ayant le format suivant :

Nom	: Blo
Adresse	: rue de la medecine
Telephone	: 418 656-2131

Pour un abonné Blo, demeurant rue de la medecine et ayant le numéro de téléphone 418 656-2131

La fonction `verifieInvariant` pour pouvoir implanter le contrat

Rappel : Complétez Abonne.h si nécessaire.

La classe `Abonne` est une classe abstraite (voir plus loin).

Pour la classe **Entreprise** (14 points)

La classe `Entreprise` est dérivée de la classe `Abonne` telle que présentée plus haut. On vous demande d'écrire les fonctions membre suivantes :

- un constructeur avec paramètres. Les paramètres sont le nom, le numéro de téléphone, l'adresse, le numéro de télécopie et l'adresse du site Web (voir l'interface pour les contraintes sur les attributs).

La fonction `verifieInvariant` pour pouvoir implanter le contrat

- La méthode `clone` qui doit être implantée **systématiquement** dans toute classe dérivée de la classe `Abonne` (sans qu'elle soit implémentée dans celle-ci) permet de faire une copie sur le monceau (tas ou heap) de l'objet courant. Ceci permet d'ajouter des Abonnés dans l'annuaire tout en autorisant le polymorphisme (dans la classe `Annuaire`)

- La fonction `reqAbonneFormate` qui doit être implantée **systématiquement** dans toute classe dérivée de la classe `Abonne`. Pensez à faire une déclaration en conséquence. Cette méthode retourne un objet string avec mise en forme ayant le format suivant :

abonne <code>Entreprise</code>	
Nom	: Blo
Adresse	: rue de la medecine
Telephone	: (418) 656-2131
Site Web	: www.joe.com
Telecopie	: 418 656-2134

[illegible]

La classe Annuaire comprend deux attributs : un nom et un vector contenant des pointeurs vers des Abonnes (qui peuvent être des objets Entreprise, Residentiel, ...) de l'annuaire. On vous demande d'écrire les fonctions membre suivantes :

- [illegible]

- Page 9/16

ajouterAbonne (

- La méthode `abonneEstDejaPresent`

- un destructeur. La méthode `size` appliquée sur un vector retourne le nombre d'éléments qu'il contient.

- une méthode `reqAnnuaireFormate` qui retourne un objet string formaté (voir le prototype dans l'interface). **Utiliser ici obligatoirement un iterator.** Le format de l'objet string doit être le suivant :

```
Liste des abonnées
Annuaire de : Quebec
=====
abonne Entreprise
Nom          : Blo
Adresse      : rue de la medecine
Telephone    : 418 656-2131
Site Web     : www.joe.com
Telecopie    : 418 656-2134
```

Dans l'exemple, l'annuaire de Quebec compte 1 abonné de catégorie Entreprise, (Pensez à utiliser une ostream)

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Rappel : Avez-vous pensé à mettre à jour toutes les interfaces ?

3. Classe AbonneDejaPresentException (3 points)

Cette classe permet de gérer l'exception de l'ajout d'un doublon d'abonnés dans l'annuaire. Cette classe hérite de `std::runtime_error` parce que cette exception est une erreur qui pourra toujours se produire, ce n'est pas une erreur de programmation.

Elle contient seulement une méthode qui est le constructeur suivant :

```
AbonneDejaPresentException (const std::string& p raison);
```

Ce constructeur ne fait qu'appeler le constructeur de la classe parent en lui passant la raison comme paramètre.

La méthode `what()` de la classe `std::exception` au haut de la hiérarchie permet d'obtenir la string qu'on a passé.

On vous demande de compléter l'interface de cette classe

```
// Fichier AbonneDejaPresentException.h
```

```
#include <stdexcept>
```

```
class AbonneDejaPresentException  
{
```

et d'écrire l'implémentation du constructeur.

```
AbonneDejaPresentException (const std::string& p_raison)
```

4. Test unitaire (12 points)

On vous demande d'implémenter le test unitaire du constructeur de la classe Abonne, ceci en utilisant les fonctionnalités offertes par le framework GoogleTest. Attention, n'oubliez pas que la classe Abonne est abstraite.

Les principales macros et assertions sont rappelées en annexe.

```
/**  
 * \file AbonneTesteur.cpp  
 * Implantation des tests unitaires de la classe Abonne  
 */
```

```
#include <gtest/gtest.h>  
#include " Abonne.h"
```

```
using namespace std;  
using namespace examenFinal;
```


Annexe

`ostringstream` fournit une interface pour manipuler des chaînes de caractères comme des flux de sortie.

membres public :

`string str ( ) const;` pour obtenir l'objet `string` associé (fonction membre publique)

La méthode `size()` appliquée sur un `vector` retourne le nombre d'éléments qu'il contient.

La méthode `begin()` appliquée sur un `vector` retourne un `iterator` pointant sur le début de celui-ci.

La méthode `end()` appliquée sur un `vector` retourne un `iterator` pointant un élément passé la fin de celui-ci.

Balises Doxygen

```
///! ... Documentation ...
\file
Permet de créer un bloc de documentation pour un fichier source ou d'en-tête.
\brief
Permet de créer une courte description dans un bloc de documentation. La description peut se faire sur une seule ligne ou plusieurs.
\author
Permet d'ajouter un nom d'auteur (ex: l'auteur d'un fichier ou d'une fonction).
\version
Permet l'ajout du numéro de version (ex: dans le bloc de documentation d'un fichier).
\date
Permet l'ajout d'une date.
\struct
Permet la création d'un bloc de documentation pour une structure.
\enum
Permet la création d'un bloc de documentation pour une énumération de constantes.
\fn
Permet la création d'un bloc de documentation pour une fonction ou méthode.
\def
Permet la création d'un bloc de documentation pour une macro ou constante symbolique.
\param
Permet d'ajouter un paramètre dans un bloc de documentation d'une fonction ou d'une macro. Cette balise permet aussi d'ajouter en option un tag pour montrer qu'il s'agit d'un argument entrant, sortant ou les deux en précisant au choix: [in], [out]
\return
Permet de décrire le retour d'une fonction ou d'une méthode.
\bug
Permet de commencer un paragraphe décrivant une bogue (ou plusieurs).
\class
Permet de créer un bloc de documentation d'une classe (C++). L'utilisation est identique à la balise \struct
\namespace
Permet de créer un bloc de documentation pour un espace de nom (C++).
```

Framework GoogleTest :

Assertions Basiques

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_TRUE(condition);</code>	<code>EXPECT_TRUE(condition);</code>	<i>condition is true</i>
<code>ASSERT_FALSE(condition);</code>	<code>EXPECT_FALSE(condition);</code>	<i>condition is false</i>

Comparaisons binaires (comparaisons entre deux valeurs v1 et v2 (valeurs comparables))

Fatal assertion	Nonfatal assertion	Verifies
ASSERT_EQ(<i>expected</i> , <i>actual</i>);	EXPECT_EQ(<i>expected</i> , <i>actual</i>);	<i>expected</i> == <i>actual</i>
ASSERT_NE(<i>val1</i> , <i>val2</i>);	EXPECT_NE(<i>val1</i> , <i>val2</i>);	<i>val1</i> != <i>val2</i>
ASSERT_LT(<i>val1</i> , <i>val2</i>);	EXPECT_LT(<i>val1</i> , <i>val2</i>);	<i>val1</i> < <i>val2</i>
ASSERT_LE(<i>val1</i> , <i>val2</i>);	EXPECT_LE(<i>val1</i> , <i>val2</i>);	<i>val1</i> <= <i>val2</i>
ASSERT_GT(<i>val1</i> , <i>val2</i>);	EXPECT_GT(<i>val1</i> , <i>val2</i>);	<i>val1</i> > <i>val2</i>
ASSERT_GE(<i>val1</i> , <i>val2</i>);	EXPECT_GE(<i>val1</i> , <i>val2</i>);	<i>val1</i> >= <i>val2</i>

Assertions d'exceptions

Fatal assertion	Nonfatal assertion	Verifies
ASSERT_THROW(<i>statement</i> , <i>exception_type</i>);	EXPECT_THROW(<i>statement</i> , <i>exception_type</i>);	<i>statement</i> throws an exception of the given type
ASSERT_ANY_THROW(<i>statement</i>);	EXPECT_ANY_THROW(<i>statement</i>);	<i>statement</i> throws an exception of any type
ASSERT_NO_THROW(<i>statement</i>);	EXPECT_NO_THROW(<i>statement</i>);	<i>statement</i> doesn't throw any exception

Test simple:

```
TEST(nom_cas_test, nom_test)
{
    ... corps de test ...
}
```

Tests utilisant un fixture, utiliser TEST_F()

Structure d'un fixture :

```
class FooTest : public ::testing::Test {
//constructeur
...
//méthodes
...
//attributs utilisés dans les tests, initialisés dans le constructeur
...
};
```

Barème

q1	12	4	4	4		
q2	4					
	6					
	7					
	9	7	2			
	14	5	2	3	4	
	25	5	5	5	4	6
q3	3	2	1			
q4	12					
	92					